0. ESQUEMA EMPRESA (hipóteses de ligação)

```
FUNCIONARIO(Pnome,Minicial,Unome,Cpf,Datanasc,
  Endereco,Sexo,Salario,Cpf_supervisor,Dnr)
DEPARTAMENTO(Dnome,Dnumero,Cpf_gerente,
  Data_inicio_gerente)
LOCALIZACAO_DEP(Dnumero,Dlocal)
PROJETO(Projnome,Projnumero,Projlocal,Dnum)
TRABALHA_EM(Fcpf,Pnr,Horas)
DEPENDENTE(Fcpf,Nome_dependente,Sexo,
  Datanasc,Parentesco)
```

Chaves de junção (decorar): F.Dnr = D.Dnumero (trabalha no depto) · D.Cpf_gerente = F.Cpf (gerencia) · F.Cpf_supervisor = S.Cpf (self join) · P.Dnum = D.Dnumero (depto controla projeto) · T.Fcpf = F.Cpf e T.Pnr = P.Projnumero (quem trabalha em qual projeto) · DEP.Fcpf = F.Cpf.

Nome completo está dividido: Pnome='Joice' AND Unome='Leite' (nunca Pnome='Joice Leite'). Texto entre **aspas simples**.

1. COMO MONTAR QUALQUER CONSULTA

Ordem lógica de execução: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → TOP. Por isso: agregação (SUM, COUNT...) **não** vai no WHERE (vai no HAVING) e alias do SELECT só vale no ORDER BY.

Roteiro: ① Quais colunas saem? ② Em que tabelas estão? ③ Como elas se ligam (FK acima)? ④ Filtro é por linha (WHERE) ou por grupo (HAVING)? ⑤ "por X / cada X" → GROUP BY X. ⑥ "todos, mesmo sem..." → LEFT JOIN. ⑦ "não possui / não é" → LEFT JOIN + IS NULL, NOT EXISTS ou EXCEPT. ⑧ comparar com valor calculado (média, máximo) → subconsulta.

2. SELECT, WHERE e filtros

Operadores: = <> > >= < <= · AND / OR / NOT · BETWEEN a AND b (**inclusivo**) · IN (lista) · LIKE: **%** = qualquer sequência, **_** = 1 caractere · IS NULL / IS NOT NULL. DISTINCT remove linhas repetidas.

```
-- homens com salário >= 30.000
SELECT * FROM FUNCIONARIO
WHERE Sexo = 'M' AND Salario >= 30000;

-- mora em SP ou Curitiba / não mora em SP
WHERE Endereco LIKE '%São Paulo%'
   OR Endereco LIKE '%Curitiba%'
WHERE Endereco NOT LIKE '%São Paulo%'

-- SEM supervisor (sem = IS NULL)
WHERE Cpf_supervisor IS NULL

-- 3 maiores salários (TOP exige ORDER BY)
SELECT TOP 3 * FROM FUNCIONARIO
ORDER BY Salario DESC;

-- nascidos em 1972 (use função de data)
WHERE YEAR(Datanasc) = 1972

-- salário 25.000 ou 30.000 / faixa
WHERE Salario IN (25000, 30000)
WHERE Dnr = 5 AND Salario BETWEEN 30000
                              AND 40000
-- alias de coluna
SELECT Pnome AS Nome, Unome AS Sobrenome
```

⚠ "= NULL" nunca é verdadeiro → sempre IS NULL. "maior ou igual" é >=, não =. "no mínimo 50" é >= 50.

3. AGREGAÇÃO

COUNT(*) conta linhas · **COUNT(col)** ignora NULL · SUM · AVG · MIN · MAX. AVG de coluna INT **trunca** → AVG(CAST(col AS DECIMAL(10,2))). Para trazer a **linha** do maior/menor, use subconsulta escalar:

```
SELECT COUNT(*) AS Total FROM FUNCIONARIO;
SELECT SUM(Salario) AS CustoMensal
FROM FUNCIONARIO;
-- funcionário com MENOR salário
SELECT * FROM FUNCIONARIO
WHERE Salario = (SELECT MIN(Salario)
                 FROM FUNCIONARIO);
-- acima da média
WHERE Salario > (SELECT AVG(Salario)
                 FROM FUNCIONARIO);
```

4. JOINS

INNER: só pares que casam. **LEFT**: todos da esquerda + NULL onde não casou. **RIGHT**: todos da direita. **FULL**: todos dos dois lados. **CROSS**: produto cartesiano (n × m linhas, sem ON). **SELF**: a mesma tabela com dois aliases.

```
-- nome/endereço de quem trabalha em Pesquisa
SELECT F.Pnome, F.Unome, F.Endereco
FROM FUNCIONARIO F
JOIN DEPARTAMENTO D ON F.Dnr = D.Dnumero
WHERE D.Dnome = 'Pesquisa';

-- quem trabalha no ProdutoX (via TRABALHA_EM!)
SELECT F.Pnome, F.Unome
FROM FUNCIONARIO F
JOIN TRABALHA_EM T ON T.Fcpf = F.Cpf
JOIN PROJETO P ON P.Projnumero = T.Pnr
WHERE P.Projnome = 'ProdutoX';

-- projetos em Mauá + depto + dados do gerente
SELECT P.Projnumero, P.Dnum, F.Unome,
       F.Endereco, F.Datanasc
FROM PROJETO P
JOIN DEPARTAMENTO D ON P.Dnum = D.Dnumero
JOIN FUNCIONARIO F ON D.Cpf_gerente = F.Cpf
WHERE P.Projlocal = 'Mauá';

-- TODOS os funcionários + depto (se houver)
SELECT F.Unome, D.Dnome
FROM FUNCIONARIO F
LEFT JOIN DEPARTAMENTO D ON F.Dnr = D.Dnumero;

-- funcionários SEM depto
... LEFT JOIN DEPARTAMENTO D
      ON F.Dnr = D.Dnumero
WHERE D.Dnumero IS NULL;

-- deptos SEM funcionário (LEFT ou RIGHT)
SELECT D.Dnome FROM DEPARTAMENTO D
LEFT JOIN FUNCIONARIO F ON F.Dnr = D.Dnumero
WHERE F.Cpf IS NULL;
-- mesmo resultado com RIGHT:
FROM FUNCIONARIO F RIGHT JOIN DEPARTAMENTO D
  ON F.Dnr = D.Dnumero WHERE F.Cpf IS NULL;

-- SELF JOIN: funcionário e seu supervisor
SELECT F.Pnome AS Funcionario,
       S.Pnome AS Supervisor
FROM FUNCIONARIO F
JOIN FUNCIONARIO S ON F.Cpf_supervisor=S.Cpf;

SELECT F.Unome, D.Dnome      -- CROSS
FROM FUNCIONARIO F CROSS JOIN DEPARTAMENTO D;
```

⚠ INNER JOIN + WHERE ... IS NULL nunca retorna nada: o "não possui" exige LEFT/RIGHT. No RIGHT JOIN, a tabela que você quer inteira fica à **direita**.

5. OPERAÇÕES DE CONJUNTO

UNION junta e remove repetidos · **UNION ALL** mantém repetidos · **EXCEPT** = está em A e não em B · **INTERSECT** = está em A e em B. Regras: mesmo nº de colunas, tipos compatíveis, nomes vêm da 1ª consulta, ORDER BY só no final.

```
-- cidades com atividade (depto ou projeto)
SELECT Dlocal AS Cidade FROM LOCALIZACAO_DEP
UNION                          -- UNION ALL: dup
SELECT Projlocal FROM PROJETO
ORDER BY Cidade;

-- todas as pessoas (colunas faltantes=NULL)
SELECT Pnome, Sexo, Datanasc FROM FUNCIONARIO
UNION
SELECT Nome_dependente, Sexo, Datanasc
FROM DEPENDENTE;

-- NÃO são gerentes
SELECT Cpf FROM FUNCIONARIO
EXCEPT
SELECT Cpf_gerente FROM DEPARTAMENTO;

-- SÃO supervisores / NÃO são supervisores
SELECT Cpf FROM FUNCIONARIO
INTERSECT            -- ou EXCEPT p/ "não são"
SELECT Cpf_supervisor FROM FUNCIONARIO;
```

⚠ NOT IN com subconsulta que contém NULL retorna **vazio** → acrescente WHERE col IS NOT NULL na subconsulta (ou use NOT EXISTS). Gerente (Cpf_gerente) ≠ supervisor (Cpf_supervisor).

6. GROUP BY e HAVING

Toda coluna do SELECT que **não** está em agregação precisa estar no GROUP BY. Agrupe pela coluna que define o "cada" — agrupar por algo a mais (ex.: Pnome) quebra os grupos. WHERE filtra linhas **antes**; HAVING filtra grupos **depois**.

```
-- qtd por sexo (mostre a coluna agrupada!)
SELECT Sexo, COUNT(*) AS Qtd
FROM FUNCIONARIO GROUP BY Sexo;

-- maior salário por depto
SELECT D.Dnome, MAX(F.Salario) AS Maior
FROM DEPARTAMENTO D
JOIN FUNCIONARIO F ON F.Dnr = D.Dnumero
GROUP BY D.Dnome;

-- projetos com no mínimo 50h no total
SELECT P.Projnome, SUM(T.Horas) AS Total
FROM PROJETO P
JOIN TRABALHA_EM T ON T.Pnr = P.Projnumero
GROUP BY P.Projnome
HAVING SUM(T.Horas) >= 50;

-- deptos com mais de 3 funcionários
... GROUP BY D.Dnome HAVING COUNT(F.Cpf) > 3;
```

Incluir grupos vazios (depto sem ninguém = 0): parta da tabela "pai" com LEFT JOIN e conte a coluna da filha: COUNT(F.Cpf), nunca COUNT(*).

7. SUBCONSULTAS: IN, EXISTS, ANY, ALL

Escalar devolve 1 valor (use com =, >). **IN** devolve uma lista (1 coluna). **EXISTS** é verdadeiro se a subconsulta retorna ≥1 linha — normalmente **correlacionada** (usa alias da consulta de fora); SELECT 1 dentro. **> ANY** = maior que pelo menos um (= > mínimo). **> ALL** = maior que todos (= > máximo). ALL com subconsulta vazia → sempre verdadeiro.

```
-- gerentes de algum depto (EXISTS)
SELECT F.* FROM FUNCIONARIO F
WHERE EXISTS (SELECT 1 FROM DEPARTAMENTO D
              WHERE D.Cpf_gerente = F.Cpf);

-- deptos com projetos
SELECT D.* FROM DEPARTAMENTO D
WHERE EXISTS (SELECT 1 FROM PROJETO P
              WHERE P.Dnum = D.Dnumero);

-- mesmo projeto E mesmas horas do Fernando
SELECT DISTINCT F.Pnome, F.Unome
FROM FUNCIONARIO F
JOIN TRABALHA_EM T ON T.Fcpf = F.Cpf
WHERE F.Cpf <> '33344555587'
  AND EXISTS (SELECT 1 FROM TRABALHA_EM X
      WHERE X.Fcpf = '33344555587'
        AND X.Pnr = T.Pnr
        AND X.Horas = T.Horas);

-- ANY: ganha mais que algum da Administração
SELECT F.* FROM FUNCIONARIO F
WHERE F.Salario > ANY (
  SELECT F2.Salario FROM FUNCIONARIO F2
  JOIN DEPARTAMENTO D ON F2.Dnr = D.Dnumero
  WHERE D.Dnome = 'Administração');

-- ALL: projetos com mais horas que TODOS
--      os projetos de Itu (somar por projeto)
SELECT P.Projnome, SUM(T.Horas) AS Total
FROM PROJETO P
JOIN TRABALHA_EM T ON T.Pnr = P.Projnumero
GROUP BY P.Projnumero, P.Projnome
HAVING SUM(T.Horas) > ALL (
  SELECT SUM(T2.Horas)
  FROM PROJETO P2
  JOIN TRABALHA_EM T2 ON T2.Pnr=P2.Projnumero
  WHERE P2.Projlocal = 'Itu'
  GROUP BY P2.Projnumero);
```

Equivalências úteis: "não possui" = NOT EXISTS = LEFT JOIN + IS NULL = EXCEPT. "> todos" = > ALL = > (SELECT MAX...). "> algum" = > ANY = > (SELECT MIN...).

8. INSERT, UPDATE, DELETE (preparação)

Exercícios de LEFT/RIGHT JOIN pedem dados "órfãos". Liste as colunas no INSERT; colunas omitidas ficam NULL (se permitirem). **UPDATE/DELETE sem WHERE** afeta a tabela inteira: rode antes um SELECT com o mesmo WHERE.

```
-- deptos novos (Cpf_gerente NOT NULL? use
-- um Cpf existente, ex. '88866555576')
INSERT INTO DEPARTAMENTO
  (Dnome, Dnumero, Cpf_gerente)
VALUES ('Marketing', 8, '88866555576'),
       ('Jurídico',  9, '88866555576');
-- funcionário SEM depto (Dnr = NULL)
INSERT INTO FUNCIONARIO
  (Pnome, Unome, Cpf, Sexo, Salario, Dnr)
VALUES ('Ana','Teste','11111111111','F',
        3000, NULL);
UPDATE FUNCIONARIO
SET Salario = Salario * 1.10
WHERE Dnr = 5;
DELETE FROM FUNCIONARIO
WHERE Cpf = '11111111111';
-- testar sem estragar a base:
BEGIN TRAN; UPDATE ...; SELECT ...; ROLLBACK;
```

9. NULL e funções úteis

Lógica de 3 valores: qualquer comparação com NULL dá **UNKNOWN**, e o WHERE só deixa passar TRUE. COUNT(col), SUM, AVG, MIN e MAX ignoram NULL. **ISNULL(x, v)** troca NULL por v · **COALESCE(a, b, c)** = 1º não nulo · **IIF(cond, se_v, se_f)** = CASE curto · **NULLIF(a, b)** = NULL se a=b (evita divisão por zero).

```
SELECT Pnome,
  ISNULL(Cpf_supervisor, 'sem supervisor')
      AS Supervisor,
  IIF(Salario > 30000, 'Alto', 'Normal')
      AS Faixa
FROM FUNCIONARIO;
-- texto
LEN(s) UPPER(s) LOWER(s) LEFT(s,3) RIGHT(s,2)
SUBSTRING(s, inicio, qtd)  REPLACE(s,'a','b')
TRIM(s)  CONCAT(a,' ',b)
-- números
ROUND(x, 2)  FLOOR(x)  CEILING(x)  ABS(x)
```

Ordem de ESCRITA (obrigatória): SELECT [DISTINCT] [TOP n] → FROM → JOIN ... ON → WHERE → GROUP BY → HAVING → ORDER BY.

10. EXEMPLO RESOLVIDO com o roteiro

"Para cada funcionário do depto Pesquisa, mostre o nome e o total de horas em projetos, só de quem soma mais de 20 h, do maior para o menor."

① Saem: nome + total de horas. ② Tabelas: FUNCIONARIO (nome), TRABALHA_EM (horas), DEPARTAMENTO (Dnome). ③ Ligações: F.Dnr=D.Dnumero e T.Fcpf=F.Cpf. ④ "do depto Pesquisa" = filtro de linha → WHERE; "soma mais de 20 h" = filtro de grupo → HAVING. ⑤ "para cada funcionário" → GROUP BY Cpf e nome. ⑥ "maior para o menor" → ORDER BY ... DESC.

```
SELECT F.Pnome, F.Unome, SUM(T.Horas) AS Total
FROM FUNCIONARIO F
JOIN DEPARTAMENTO D ON F.Dnr = D.Dnumero
JOIN TRABALHA_EM T  ON T.Fcpf = F.Cpf
WHERE D.Dnome = 'Pesquisa'
GROUP BY F.Cpf, F.Pnome, F.Unome
HAVING SUM(T.Horas) > 20
ORDER BY Total DESC;
```

11. LOTES, VARIÁVEIS e PRINT

GO termina um lote: variáveis **morrem** no GO e CREATE FUNCTION/PROCEDURE tem que ser o **primeiro** comando do lote (coloque GO antes e depois). **DECLARE @x TIPO;** — valor inicial NULL. **SET @x = valor** (um valor). **SELECT @x = col FROM ... WHERE ...** (busca da tabela; se vierem várias linhas, fica a última). Qualquer conta ou concatenação com NULL vira NULL → PRINT sai vazio.

```
GO
DECLARE @nome VARCHAR(50) = 'Jennifer',
        @salario DECIMAL(10,2),
        @novo    DECIMAL(10,2);

SELECT @salario = Salario
FROM FUNCIONARIO WHERE Pnome = @nome;

SET @novo = @salario * 1.10;     -- +10%
PRINT 'Salário de ' + @nome + ' atualizado = '
      + CAST(@novo AS VARCHAR(20));
GO
```

Percentuais: +5% = ×1.05 · +10% = ×1.10 · −20% = ×0.80 (×1.5 é +50%!).

12. DATAS

```
GETDATE()                    -- data/hora atual
YEAR(d)  MONTH(d)  DAY(d)
DATEDIFF(YEAR, @nasc, GETDATE())  -- idade*
DATEDIFF(MONTH, @admissao, GETDATE()) -- meses
DATEADD(YEAR, 65, @nasc)    -- soma intervalo
```

*DATEDIFF(YEAR) conta viradas de ano: se o aniversário ainda não chegou, dá 1 a mais. Idade exata: SET @idade = DATEDIFF(DAY, @nasc, GETDATE()) / 365.25 (ao guardar em INT, a parte decimal é descartada).

```
GO
DECLARE @nasc DATE, @idade INT;
SELECT @nasc = Datanasc FROM FUNCIONARIO
WHERE Pnome = 'Jennifer';
SET @idade = DATEDIFF(YEAR, @nasc, GETDATE());
PRINT 'Idade: ' + CAST(@idade AS VARCHAR(3));
```

13. CAST, CONVERT e concatenação

PRINT e o operador + com texto exigem **string**. CAST(valor AS tipo) é padrão SQL; CONVERT(tipo, valor, estilo) é do SQL Server e aceita **estilo** de data. CONCAT(a, b, ...) converte sozinho e trata NULL como ''.

```
-- 'O funcionário(a) Jennifer tem um
--  salário de : R$ 43000.'
PRINT 'O funcionário(a) ' + @nome
 + ' tem um salário de : R$ '
 + CAST(CAST(@salario AS INT) AS VARCHAR(10))
 + '.';
-- com CONVERT
+ CONVERT(VARCHAR(10), CONVERT(INT, @salario))

-- data no padrão brasileiro dd/mm/aaaa
CONVERT(VARCHAR(10), @nasc, 103)
-- 101 mm/dd/aaaa · 112 aaaammdd · 120 ISO
CONCAT(Pnome, ' ', Unome) AS NomeCompleto
```

14. IF / ELSE e CASE

IF controla o fluxo do script (fora do SELECT). Mais de um comando no bloco → **BEGIN ... END**. **CASE** gera valor **dentro** do SELECT: testa na ordem, para no primeiro WHEN verdadeiro, ELSE opcional (senão NULL).

```
GO -- abaixo da média?
DECLARE @nome VARCHAR(50) = 'Ana',
        @sal DECIMAL(10,2), @media DECIMAL(10,2);
SELECT @media = AVG(Salario) FROM FUNCIONARIO;
SELECT @sal = Salario FROM FUNCIONARIO
WHERE Pnome = @nome;
IF @sal < @media
   PRINT @nome + ' recebe abaixo da média';
ELSE
   PRINT @nome + ' recebe acima da média';

-- perto da aposentadoria (60 anos)
IF DATEDIFF(YEAR, @nasc, GETDATE()) >= 55
BEGIN
   PRINT 'Próximo da aposentadoria';
   PRINT 'Faltam ' + CAST(60 - DATEDIFF(YEAR,
         @nasc, GETDATE()) AS VARCHAR) + ' anos';
END
ELSE PRINT 'Longe da aposentadoria';

-- novo contratado (últimos 6 meses)
IF DATEDIFF(MONTH, @admissao, GETDATE()) <= 6
   PRINT 'Novo contratado';

-- CASE: faixas sem sobreposição
SELECT Pnome, Salario,
  CASE WHEN Salario <= 10000 THEN 'Baixo'
       WHEN Salario <= 30000 THEN 'Médio'
       ELSE 'Alto' END AS Categoria
FROM FUNCIONARIO;

-- CASE com agregação: tamanho do depto
SELECT D.Dnome, COUNT(F.Cpf) AS Qtd,
  CASE WHEN COUNT(F.Cpf) <= 1 THEN 'pequeno'
       WHEN COUNT(F.Cpf) <= 3 THEN 'médio'
       ELSE 'grande' END AS Tamanho
FROM DEPARTAMENTO D
LEFT JOIN FUNCIONARIO F ON F.Dnr = D.Dnumero
GROUP BY D.Dnumero, D.Dnome;
```

15. WHILE

Repete enquanto a condição for verdadeira. **Sempre** altere a variável da condição dentro do laço (senão, loop infinito). BREAK sai do laço; CONTINUE pula para o próximo teste. Alterar a variável não altera a tabela: precisa de UPDATE.

```
GO -- +5% até chegar em 30.000
DECLARE @sal DECIMAL(10,2);
SELECT @sal = Salario FROM FUNCIONARIO
WHERE Pnome = 'Joice' AND Unome = 'Leite';

WHILE @sal < 30000
BEGIN
   SET @sal = @sal * 1.05;
   PRINT 'Novo salário: '
         + CAST(@sal AS VARCHAR(20));
END

UPDATE FUNCIONARIO SET Salario = @sal
WHERE Pnome = 'Joice' AND Unome = 'Leite';
```

16. FUNÇÕES (UDF) — passo a passo

① GO antes. ② CREATE [OR ALTER] FUNCTION **dbo.**nome(@param TIPO). ③ RETURNS **tipo** (escalar) ou RETURNS **TABLE** (tabela). ④ Corpo. ⑤ GO depois. ⑥ Chamar. Função **não** pode INSERT/UPDATE/DELETE em tabelas, nem PRINT, nem TRY/CATCH; só calcula e devolve.

a) Escalar → devolve 1 valor · chamada: SELECT dbo.fn(...)

```
GO
CREATE OR ALTER FUNCTION dbo.fn_NomeSupervisor
   (@cpf CHAR(11))
RETURNS VARCHAR(100)
AS
BEGIN
   DECLARE @nome VARCHAR(100);
   SELECT @nome = CONCAT(S.Pnome,' ',S.Unome)
   FROM FUNCIONARIO F
   JOIN FUNCIONARIO S ON S.Cpf=F.Cpf_supervisor
   WHERE F.Cpf = @cpf;
   RETURN @nome;        -- RETURN é obrigatório
END
GO
SELECT dbo.fn_NomeSupervisor('12345678966');
-- dentro de consulta, linha a linha:
SELECT Pnome, dbo.fn_NomeSupervisor(Cpf)
FROM FUNCIONARIO;

GO -- verdadeiro/falso → BIT (1/0)
CREATE OR ALTER FUNCTION
  dbo.fn_GanhaMaisQueSupervisor(@cpf CHAR(11))
RETURNS BIT
AS
BEGIN
   DECLARE @f DECIMAL(10,2), @s DECIMAL(10,2);
   SELECT @f = F.Salario, @s = S.Salario
   FROM FUNCIONARIO F
   JOIN FUNCIONARIO S ON S.Cpf=F.Cpf_supervisor
   WHERE F.Cpf = @cpf;
   IF @f > @s RETURN 1;
   RETURN 0;
END
GO
-- escalar só com parâmetro
CREATE OR ALTER FUNCTION
  dbo.fn_AnoAposentadoria(@nasc DATE)
RETURNS INT
AS BEGIN RETURN YEAR(@nasc) + 65; END
GO
```

b) Tabela inline → 1 SELECT · chamada: SELECT * FROM dbo.fn(...)

```
GO
CREATE OR ALTER FUNCTION
  dbo.fn_ProjetosPorFuncionario(@cpf CHAR(11))
RETURNS TABLE
AS
RETURN (                  -- sem BEGIN/END
   SELECT P.Projnome
   FROM PROJETO P
   JOIN TRABALHA_EM T ON P.Projnumero = T.Pnr
   WHERE T.Fcpf = @cpf
);
GO
SELECT * FROM
  dbo.fn_ProjetosPorFuncionario('12345678966');
```

c) Tabela multi-instrução → monta uma variável-tabela

```
CREATE OR ALTER FUNCTION dbo.fn_Dependentes
  (@cpf CHAR(11))
RETURNS @r TABLE (Nome VARCHAR(50),
                  Parentesco VARCHAR(20))
AS
BEGIN
   INSERT INTO @r
   SELECT Nome_dependente, Parentesco
   FROM DEPENDENTE WHERE Fcpf = @cpf;
   RETURN;                -- RETURN vazio
END
```

17. STORED PROCEDURES — passo a passo

① GO antes. ② CREATE [OR ALTER] PROCEDURE nome. ③ Parâmetros: @p TIPO, opcional com **= valor padrão**, saída com **OUTPUT**. ④ AS BEGIN ... END. ⑤ GO. ⑥ EXEC nome valor1, valor2 ou EXEC nome @p = valor. Apagar: DROP PROCEDURE nome / DROP FUNCTION dbo.nome.

Função × Procedure: função devolve valor/tabela, é usada **dentro** do SELECT e não altera dados. Procedure é chamada com **EXEC**, pode INSERT/UPDATE/DELETE, transação, TRY/CATCH, PRINT, vários SELECTs e parâmetros OUTPUT; não entra no SELECT.

Biblioteca (hipótese): Categoria(ID, Tipo_categoria) · Autor(ID, Nome, Nacionalidade) · Editora(ID, Nome) · Livro(ISBN, Titulo, Ano, Fk_editora, Fk_categoria) · LivroAutor(Fk_livro, Fk_autor).

INSERT (categoria, autor, editora)

```
GO
CREATE OR ALTER PROCEDURE usp_InserirAutor
   @Nome VARCHAR(100), @Nac VARCHAR(50)
AS
BEGIN
   INSERT INTO Autor (Nome, Nacionalidade)
   VALUES (@Nome, @Nac);  -- ID é IDENTITY
END
GO
EXEC usp_InserirAutor 'Machado', 'Brasileira';
```

UPDATE com parâmetros opcionais

```
CREATE OR ALTER PROCEDURE usp_AtualizarLivro
   @ISBN VARCHAR(20),
   @Titulo VARCHAR(255) = NULL,
   @Ano INT = NULL
AS
BEGIN
   UPDATE Livro
   SET Titulo = ISNULL(@Titulo, Titulo),
       Ano    = ISNULL(@Ano, Ano)
   WHERE ISBN = @ISBN;
   IF @@ROWCOUNT = 0 PRINT 'ISBN não existe';
END
GO
EXEC usp_AtualizarLivro @ISBN='978...', @Ano=2005;
```

DELETE em 2 tabelas → transação (filha antes do pai)

```
CREATE OR ALTER PROCEDURE usp_ExcluirAutor
   @ID INT
AS
BEGIN
   BEGIN TRY
      BEGIN TRANSACTION;
        DELETE FROM LivroAutor WHERE Fk_autor=@ID;
        DELETE FROM Autor      WHERE ID = @ID;
      COMMIT;
   END TRY
   BEGIN CATCH
      ROLLBACK;
      PRINT 'Erro: ' + ERROR_MESSAGE();
   END CATCH
END
-- excluir editora: UPDATE Livro
--   SET Fk_editora = NULL WHERE Fk_editora=@ID;
--   depois DELETE FROM Editora WHERE ID=@ID;
```

SELECT com parâmetro (consultas da lista)

```
CREATE OR ALTER PROCEDURE usp_LivrosPorAutor
   @Nome VARCHAR(100)
AS
BEGIN
   SELECT L.*
   FROM Livro L
   JOIN LivroAutor LA ON LA.Fk_livro = L.ISBN
   JOIN Autor A       ON A.ID = LA.Fk_autor
   WHERE A.Nome LIKE '%' + @Nome + '%';
END
-- mesma forma: por categoria (JOIN Categoria),
-- por editora, por ano (WHERE Ano=@Ano),
-- faixa: WHERE ISBN BETWEEN @Ini AND @Fim,
-- nacionalidade: SELECT DISTINCT L.* ...
```

Padrões de consulta dentro da procedure

```
-- livros por categoria (inclui categoria vazia)
SELECT C.Tipo_categoria, COUNT(L.ISBN) AS Qtd
FROM Categoria C
LEFT JOIN Livro L ON L.Fk_categoria = C.ID
GROUP BY C.ID, C.Tipo_categoria;
-- mais de um autor
SELECT L.ISBN, L.Titulo FROM Livro L
JOIN LivroAutor LA ON LA.Fk_livro = L.ISBN
GROUP BY L.ISBN, L.Titulo HAVING COUNT(*) > 1;
-- sem autor
SELECT * FROM Livro L WHERE NOT EXISTS
 (SELECT 1 FROM LivroAutor LA
  WHERE LA.Fk_livro = L.ISBN);
-- sem editora
SELECT * FROM Livro WHERE Fk_editora IS NULL;
-- associar sem duplicar
IF NOT EXISTS (SELECT 1 FROM LivroAutor
   WHERE Fk_livro=@ISBN AND Fk_autor=@IDAutor)
   INSERT INTO LivroAutor VALUES(@ISBN,@IDAutor);
```

Parâmetro OUTPUT (ano médio por categoria)

```
CREATE OR ALTER PROCEDURE usp_AnoMedio
   @Cat VARCHAR(100),
   @Media DECIMAL(6,1) OUTPUT
AS
BEGIN
   SELECT @Media = AVG(CAST(L.Ano AS DECIMAL(6,1)))
   FROM Livro L
   JOIN Categoria C ON L.Fk_categoria = C.ID
   WHERE C.Tipo_categoria = @Cat;
END
GO
DECLARE @m DECIMAL(6,1);
EXEC usp_AnoMedio 'Romance', @m OUTPUT;
PRINT @m;
```

18. CHECKLIST ANTES DE ENTREGAR

☐ Filtro de texto com aspas simples e nome dividido (Pnome/Unome). ☐ "sem/não possui" → IS NULL / NOT EXISTS, nunca = NULL. ☐ LEFT JOIN quando a pergunta diz "todos". ☐ Cada coluna não agregada do SELECT está no GROUP BY? ☐ Condição de agregação no HAVING. ☐ >= para "no mínimo / maior ou igual". ☐ TOP acompanhado de ORDER BY. ☐ AVG de inteiro com CAST. ☐ Variável recebeu valor antes de usar? ☐ CAST/CONVERT antes de concatenar número com texto. ☐ GO antes e depois de CREATE FUNCTION/PROCEDURE. ☐ Função escalar chamada com dbo. ☐ WHILE altera a variável da condição. ☐ DELETE: filha (LivroAutor) antes do pai.